Arduino based Obstacle avoiding and line following Car

A

Project Exhibition –1

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted to

VIT BHOPAL UNIVERSITY (M.P.)

Submitted by

TANYA TANU (24BEC10116)
PAPA KARTHIK REDDY (24BEC10157)
ADDEPALLI SAI KRISHNAM RAJU (24BEC10165)

Under the Supervision of
Dr. MONICA P

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING
VIT BHOPAL UNIVERSITY

BHOPAL (M.P.)-466114

September -2025

VIT BHOPAL UNIVERSITY BHOPAL (M.P.) 466114

SCHOOL OF ELECTRICAL & ELECTRONICS ENGG.

CANDIDATE'S DECLARATION

I hereby declare that the Dissertation entitled Arduino based obstacle avoiding and line following car is my own work conducted under the supervision of Dr. Monica P Assistant Professor, Electronics and Communication Engineering at VIT University, Bhopal.
I further declare that to the best of my knowledge this report does not contain any part of work that has been submitted for the award of any degree either in this university or in other university / Deemed University without proper citation.

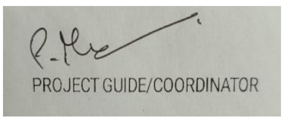
TANYA TANU (24BEC10116)
PAPA KARTHIK REDDY (24BEC10157)
ADDEPALLI SAI KRISHNAM RAJU (24BEC10165)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date: 29th September 2025

Guide Name: Dr. Monica P

Designation: Assistant Professor



Digital Signature of Guide

VIT UNIVERSITY BHOPAL (M.P.) – 466114

SCHOOL OF ELECTRICAL & ELECTRONICS ENGG.

CERTIFICATE

This is to certify that the work embodied in this Project Exhibition -1 report entitled **"Arduino based obstacle avoiding and line following car"** has been satisfactorily completed by **Tanya Tanu (24BEC10116), Papa Karthik Reddy (24BEC10157), Addepalli Sai Krishnam Raju (24bec10165)** in the School of Electrical & Electronics Engineering of Electronics and communication Engineering at VIT University, Bhopal. This work is a bonafide piece of work, carried out under my/our guidance in the School of Electrical and Electronics Engineering for the partial fulfilment of the degree of Bachelor of Technology.

Dr. Monica P
(Assistant Professor)

Forwarded by

Dr. Sivasankaran V
(Program chair)

Approved by

Dr. Suresh M
(Associate professor and Dean)

ACKNOWLEDGEMENT

I wish to express my heartfelt gratitude to **Dr. Sivasankaran V** , Program chair of the Department, School of Electrical & Electronics Engineering for much of his valuable support encouragement in carrying out this work.

I would like to express my deepest gratitude and sincere thanks to my project guide, **Dr. Monica P**, whose expert guidance, valuable suggestions, and continuous encouragement have been instrumental in the successful completion of this project. Their insightful feedback and unwavering support helped me navigate the challenges faced during the research and development phases.

I am also profoundly thankful to the faculty members and staff of SEEE, VIT Bhopal University, for providing the necessary resources, technical assistance, and an inspiring academic environment that facilitated my work. Their kind cooperation made a significant impact on the quality of this project. Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

Executive Summary

This project presents the design and development of an Arduino-based Obstacle Avoiding and Line Following Car, an autonomous robotic prototype capable of navigating along a defined path while avoiding unexpected obstacles. The motivation behind this project lies in the increasing demand for automation in industries, logistics, and transportation, where autonomous systems improve efficiency, reduce human effort, and enhance safety.

The proposed system integrates two major functionalities: line following, achieved using Infrared (IR) sensors, and obstacle avoidance, implemented through an Ultrasonic (HC-SR04) sensor. An Arduino Uno microcontroller acts as the central controller, processing sensor inputs and generating motor control signals through the L293D motor driver. The car can follow a black line on a white surface with high accuracy and successfully detect and bypass obstacles within a 10–30 cm range.

The methodology involves hardware assembly, algorithm design, and Arduino IDE programming. Testing was carried out on tracks with straight, curved, and intersecting paths, along with obstacle scenarios. Results demonstrated reliable line following performance and effective obstacle detection and avoidance.

This project contributes a low-cost, scalable, and beginner-friendly robotic model suitable for educational demonstrations, research in robotics, and industrial applications such as warehouse automation and autonomous delivery systems. Future extensions could include IoT integration, machine learning–based navigation, GPS for outdoor environments, and camera-based vision systems for advanced robotics applications.

List of Figures

SL.NO	FIGURE NAME	FIGURE NUMBER	PAGE.NO
1	Circuit Diagram	3.1	7
2	Working Principle	3.3	10
3	Ultrasonic Sensor	4.2	18
4	Testing & Result	5.1	20

LIST OF TABLES

SL.NO	NAME OF THE TABLE	TABLE NUMBER	PAGE NO
1	Uses of components	1.4	3
2	Hardware component types	3.2	9

List of Symbols & Abbreviations

- **µC**: Microcontroller
- **µP**: Microprocessor
- **IR**: Infrared
- **DC**: Direct Current
- **AC**: Alternating Current
- **Li-ion**: Lithium-ion
- **LED**: Light Emitting Diode
- **LDR**: Light Dependent Resistor
- **IDE**: Integrated Development Environment
- **I/O**: Input/Output
- **mAh**: Milliampere-hour (a unit of electric charge)
- **GPS**: Global Positioning System
- **IoT**: Internet of Things
- **AI**: Artificial Intelligence
- **PWM**: Pulse Width Modulation
- **PCB**: Printed Circuit Board
- **USB**: Universal Serial Bus

Table of Contents

CANDIDATE’S DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
EXECUTIVE SUMMARY	iv
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF SYMBOLS & ABBREVIATIONS	vii

SL.NO	CHAPTERS	Sub-Topics	PAGE.NO
1	INTRODUCTION	1.1 Motivation 1.2 Problem Definition 1.3 Objective 1.4 Methodology	1-3
2	LITERATURE REVIEW	2.1 Previous Works 2.2 Gaps Identified 2.3 Relevance of current project	4-6
3	PROBLEM FORMULATION AND PROPOSED METHODOLOGY	3.1 Circuit diagram 3.2 Hardware components 3.3 Working principle	7-10
4	IMPLEMENTATION	4.1 Hardware setup 4.2 Software/Algorithm	11-18
5	RESULTS &	5.1 Testing	19-21

	DISCUSSION	5.2 Performance Analysis 5.3 Discussion	
6	CONCLUSION & FUTURE SCOPE	6.1 Conclusion 6.2 Future Scope	21-23
7	REFERENCES	-	24

Chapter 1: INTRODUCTION

1.1 Motivation

In the modern world, automation and robotics are no longer just futuristic concepts; they are fundamental tools transforming how we live and work. Their significance stems from their ability to take over tasks that are repetitive, dangerous, or require extreme precision. By deploying these technologies, businesses and individuals can drastically reduce their reliance on manual human effort, freeing people to focus on more complex and creative challenges.

This shift delivers two major benefits: improved efficiency and enhanced accuracy. Machines can work continuously, 24/7, without getting tired or making mistakes, which is a game-changer for production lines and logistics. For instance, in an industrial setting, a robotic arm can perform the same weld thousands of times with flawless consistency, while an automated sorting system in a warehouse can accurately process thousands of packages per hour. This level of speed and precision is nearly impossible to achieve with human labor alone.

1.2 Problem Definition

While many robotic cars are designed with a singular purpose, such as just line following for assembly lines or just obstacle avoidance for navigating open spaces, this specialized approach significantly limits their usefulness. A robot that can only follow a line will be rendered useless by an unexpected box in its path, while one that only avoids obstacles might wander aimlessly without a clear objective.

A **more effective and versatile approach** is to create a single, integrated system capable of both tasks. This solution offers a more comprehensive and reliable platform. By combining the precision of line following with the situational awareness of obstacle avoidance, the robot can handle dynamic and unpredictable environments, making it far more practical for real-world scenarios.

1.3 Objectives

- **Path Following**: The fundamental objective is for the robot to exhibit accurate path-following. This involves more than simply moving forward; the robot must use sensors, such as infrared (IR) or vision-based cameras, to detect and interpret a designated line or path. Its control system must then translate this sensor data into precise motor commands to ensure it stays on the line. This capability is especially critical on curved or winding paths, where the robot must continuously adjust its steering to prevent any deviation.
- **Obstacle Avoidance**: A second critical objective is proactive obstacle avoidance. This requires the robot to have a "sense" of its surroundings. Using sensors like ultrasonic, infrared, or lidar, the robot must be able to detect any objects in its immediate path. Once an obstacle is identified within a predefined safe distance, the robot's logic must execute a decisive maneuver. This could involve stopping completely, making a sharp turn to the side, or navigating in a wide arc around the object
- **Integration**: The most advanced objective is the seamless integration of both path-following and obstacle-avoidance into a single, cohesive system. This requires a hierarchical control logic where safety is the highest priority. The robot's programming must establish a clear protocol: it should default to line-following mode, but if an obstacle is detected, this command is immediately superseded by the obstacle-avoidance routine. The system must be able to switch between these modes instantaneously and flawlessly. For example, a common logical flow is: "If an obstacle is detected, stop following the line and execute the avoidance maneuver. Once the path is clear, resume line-following."
- **Reliability**: Finally, the system must demonstrate robust and reliable performance under various conditions. A truly effective robot is not a one-trick pony; it must be able to perform its tasks consistently, regardless of minor environmental changes. This includes adapting to different types of lines (e.g., varying thickness or color), different ambient lighting that can affect sensors, and a range of obstacle types (e.g., small objects, reflective surfaces). The ultimate goal is to create a system that can be deployed in a real-world scenario with confidence, knowing it will consistently and safely perform its duties.

1.4 Methodology

1. Study of sensors and Arduino platform.
2. Hardware design: DC motors, motor driver, IR & ultrasonic sensors.
3. Software design: Arduino IDE coding in embedded C.
4. Testing on straight and curved tracks with obstacles.
5. Analysis of accuracy, speed, and reliability.

Table 1.4

Uses of the Components

SL.NO	COMPONENTS	USES
1	Arduino	Controller
2	IR-Sensor	Line Detection
3	Ultrasonic sensor	Obstacle Detection
4	Motor Driver	Motor Control
5	DC motor & wheels	Movement, Propulsion
6	Power supply	Energy Source

Chapter 2: LITERATURE REVIEW

2.1 Previous Work

- **Mahor et al. (2010): The Challenge of Environmental Interference**

  This study focused on building an obstacle-avoiding robot using infrared (IR) sensors. While the system proved effective in controlled indoor environments, it was highly susceptible to external light sources. Strong ambient light, particularly sunlight, contains infrared radiation that can interfere with the sensor's readings. This interference can cause the sensor to malfunction, leading to a failure to detect obstacles or, conversely, to register "ghost" obstacles where none exist. This design limitation makes the robot unreliable for outdoor applications or in areas with varying light conditions.

- **Kumar et al. (2015): The Gap in Situational Awareness**

  Kumar and his colleagues successfully developed highly precise line-following robots for industrial automation. Their work demonstrated the capability for accurate and efficient navigation along a predetermined path. However, their systems were designed as single-function robots and lacked any mechanism for obstacle detection. In a real-world factory environment, which is not a static space, this is a significant safety and functional drawback. The robots would be unable to stop or reroute if a person, a forklift, or a misplaced item were to block their path, posing a risk of collision and operational failure.

- **Sharma et al. (2018): The Lack of Path Guidance**

  This research implemented ultrasonic sensors to create autonomous robots capable of navigating and avoiding obstacles in open spaces. The ultrasonic technology proved effective for detecting objects and maintaining a safe distance. However, their system was not designed with a line-following capability. The robots could operate in an unstructured environment but had no means of following a precise, predefined route. This limitation makes the design unsuitable for applications that require both collision avoidance and adherence to a specific track, such as delivery robots or warehouse vehicles.

- **Singh et al. (2020): The Absence of True Autonomy**

  The project by Singh et al. created Bluetooth-controlled smart cars. While an interesting and accessible project, it did not achieve true autonomy. The cars relied on constant human control via a smartphone or remote, meaning they could not operate independently. This design is fundamentally different from a fully autonomous robot that can make its own navigation decisions. The need for continuous manual operation defeats the core purpose of automation, which is to reduce human effort and enable devices to work reliably on their own.

2.2 Gaps Identified

1. Lack of Integration Between Core Functions

A significant drawback in many robotic designs is the failure to integrate fundamental functions like line following and obstacle avoidance. Existing research often treats these as separate, mutually exclusive capabilities. This creates a critical weakness: a robot that excels at following a path is rendered useless by an unexpected obstacle, while a robot that can avoid obstacles lacks the guidance to follow a specific route. For a robot to be truly useful in dynamic, real-world scenarios—such as navigating a warehouse or a city street—it must possess an intelligent control system that can seamlessly switch between these two modes.

2. Over-reliance on Single-Function Designs

As a direct result of the lack of integration, the majority of commercially and academically available robots are built for a single purpose. They are either designed to operate solely on a predetermined track or to navigate freely in an open area. This limited functionality restricts their use to highly specific, controlled environments. The demand, however, is for versatile robots that can handle complex tasks in semi-structured settings, where a predefined path (like an assembly line) might be interrupted by an unexpected event (like a person crossing its path).

3. Limited Availability of Accessible Solutions

The third major gap is the shortage of solutions that are simultaneously cost-effective, beginner-friendly, and scalable. High-end integrated systems are often prohibitively expensive, making them inaccessible for educational purposes, small-scale prototyping, and individual hobbyists. Furthermore, many sophisticated designs require advanced programming knowledge and complex hardware configurations, posing a steep learning curve for newcomers. A truly impactful solution would be one that uses affordable, off-the-shelf components, is simple to program and assemble, and has a modular design that allows for easy expansion and customization, thereby encouraging wider adoption and innovation.

2.3 Relevance of Current Project

This project addresses the above gaps by integrating both obstacle avoidance and line following into one car using Arduino, making it suitable for academic, research, and real-world applications.

Chapter 3: PROBLEM FORMULATION AND PROPOSED METHODOLOGY

3.1 Circuit Diagram

IR Sensors + Ultrasonic Sensor → Arduino Uno → Motor Driver (L293D) → DC Motors → Robotic Car Movement

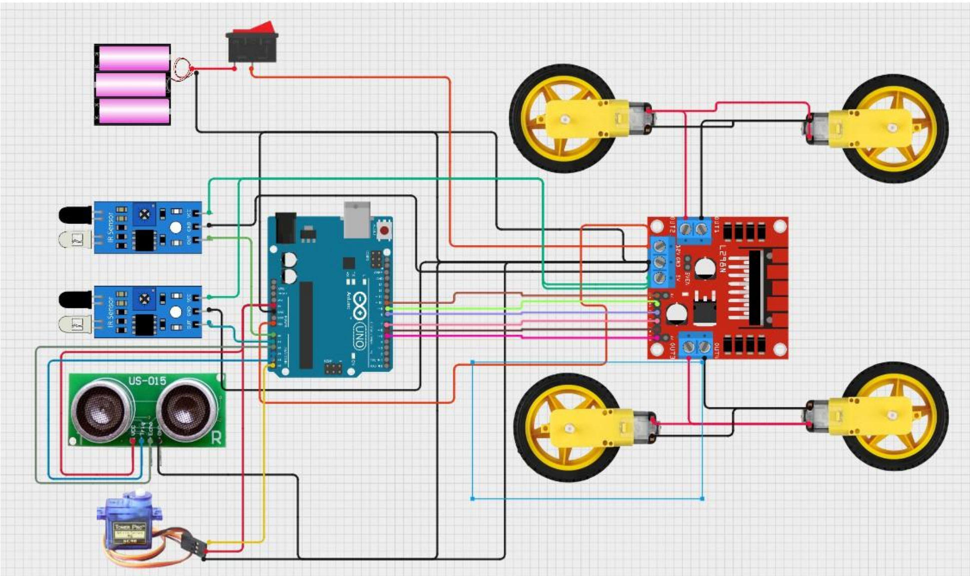


Fig no 3.1

Circuit Diagram

3.2 Hardware Components

- **Arduino Uno:** This serves as the **central processing unit** or "brain" of the robot. It is a microcontroller board that reads the input signals from all the sensors and executes the programmed instructions. It's responsible for processing the data and then sending the correct commands to the motor driver to control the robot's movement.

- **Ultrasonic Sensor (HC-SR04):** This sensor is used for **obstacle detection**. It works by emitting a high-frequency sound pulse and measuring the time it takes for the echo to return. The Arduino uses this time to calculate the distance to any object in front of the robot. This allows the car to sense and react to obstacles before colliding with them

- **IR Sensors:** These are crucial for the **line-following capability**. They are typically placed on the bottom of the robot and work by emitting a beam of infrared light. The sensors measure how much of this light is reflected back. A dark surface (like a black line) absorbs more light, while a light surface (like a white floor) reflects more. This contrast allows the Arduino to determine the robot's position relative to the line..

- **L293D Motor Driver:** The motor driver acts as an **interface or bridge** between the Arduino and the DC motors. The low-current signals from the Arduino are not powerful enough to directly run the motors. The L293D chip takes these signals and uses an external power source to provide the higher current needed to drive the motors, while also allowing the Arduino to control their direction and speed.

- **DC Motors & Wheels:** These are the **actuators** that provide movement. The DC motors convert electrical energy from the power supply into the rotational motion that turns the wheels. The Arduino, through the motor driver, controls these motors to make the robot move forward, backward, or turn..

- **Chassis:** The chassis is the **physical base frame** of the robot. It provides a sturdy structure for mounting all the components—the Arduino, sensors, motor driver, motors, and battery. The design and material of the chassis are important for the robot's stability and durability.

- **Power Supply:** A reliable power supply, in this case, **rechargeable Li-ion batteries**, is essential to power the entire system. It provides the electrical energy needed for the Arduino to operate, the sensors to take readings, and the motors to move the robot.

Table 3.2

HARDWARE COMPONENT TYPES

SL.NO	HARDWARE COMPONENTS	TYPES
1	Arduino	UNO
2	Ultrasonic Sensor	HC-SR04
3	IR Sensors	-
4	Motor Driver	L293D
5	DC Motors & Wheels	-
6	Chassis	physical base frame
7	Power Supply	rechargeable Li-ion batteries

3.3 Working Principle

1. **Line Following**: The robot uses infrared (IR) sensors to follow a black line on a white surface. The sensors detect the contrast between the two colors.

2. **Obstacle Avoiding Mode:** Ultrasonic sensor continuously measures distance. If an obstacle is within a threshold (e.g., 15 cm), Arduino redirects the car.

3. **Integration:** The car first follows the line. If an obstacle is detected, it overrides line-following temporarily, avoids it, and then resumes following the line.

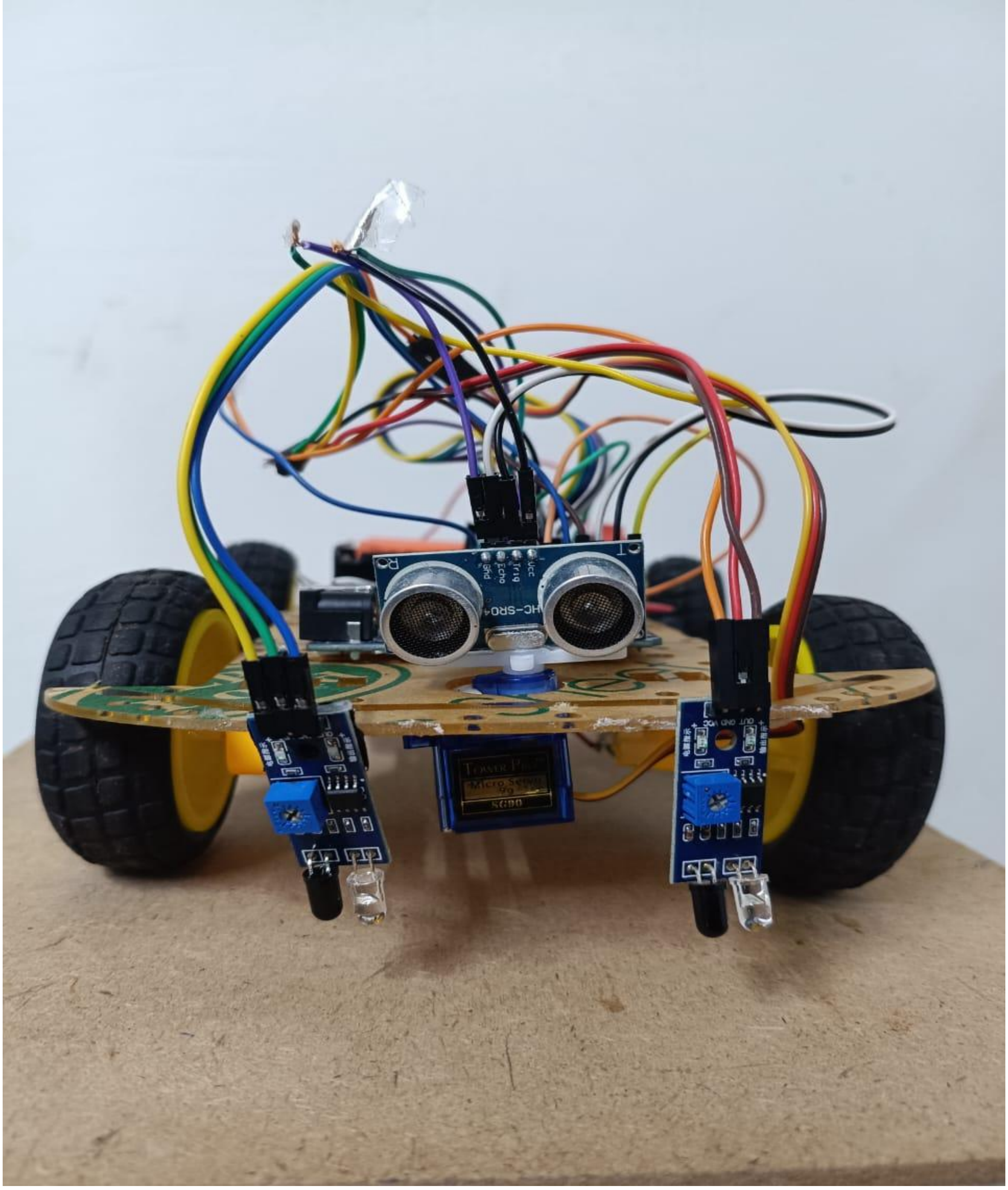


Fig no -3.3

Working Principle

Chapter 4: IMPLEMENTATION

4.1 Hardware Setup

- The sensors are strategically placed to perform their specific functions. The ultrasonic sensor is mounted at the front of the robot. This position gives it a clear field of view to accurately measure the distance to any obstacles ahead, which is essential for proactive avoidance. The IR sensors are positioned on the bottom of the chassis, facing the ground. This allows them to effectively detect the contrast between the black line and the white surface, enabling the robot to follow its designated path.

- The Arduino Uno is placed centrally on the chassis to serve as the main control hub. This central location helps with weight distribution, ensuring the robot is balanced. More importantly, it provides a convenient point for all the wiring to converge, making it easier to connect the power supply, sensors, and motor driver, which in turn simplifies the overall assembly.

- The L293D motor driver is connected to two independent DC motors. This setup is what gives the robot its steering capability. By controlling the voltage to each motor separately, the Arduino can make the robot move forward, turn left, or turn right. For instance, to turn left, the right motor can be powered while the left motor is slowed down or stopped. The entire system is powered by a battery supply that is connected via an ON/OFF switch. This switch provides a simple and critical way to manage the power flow, allowing the user to safely turn the robot on or off.

4.2 Software / Algorithm

- Program written in Arduino IDE using Embedded C.

The Main Logic Flow

The robot's program operates in a continuous loop, prioritizing safety while staying on its intended path.

- **Read IR Sensors → Follow Line**: In its default state, the robot constantly reads the data from its infrared (IR) sensors positioned on the bottom. The program is coded to interpret these readings to follow a line. For example, if the sensor on the left side detects the black line, the robot's logic will command it to turn left to stay on track. If the right sensor detects the line, it turns right. This continuous feedback loop ensures the robot maintains a precise course.

- **Continuously Check Ultrasonic Distance**: Simultaneously, and without interrupting the line-following process, the program constantly queries the ultrasonic sensor to measure the distance to any objects in front of it. This provides a real-time safety check, acting as a constant lookout for potential collisions.
- **If Obstacle Detected → Avoidance Maneuver**: This is the crucial step where the robot's integrated functionality becomes apparent. If the ultrasonic sensor detects an obstacle within a pre-programmed distance threshold (e.g., 15 cm), the line-following logic is immediately overridden. The robot's control system executes a pre-defined set of commands to avoid the object, which could include stopping, turning a specific angle (e.g., 90 degrees), moving forward slightly, and then turning back to re-engage with the line. Once the avoidance maneuver is complete, the robot returns to the primary line-following mode.

Arduino Code:

```
#define enA 10//Enable1 L298 Pin enA
#define in1 9 //Motor1  L298 Pin in1 #define
in2 8 //Motor1  L298 Pin in1 #define in3 7
//Motor2  L298 Pin in1 #define in4 6 //Motor2
L298 Pin in1 #define enB 5 //Enable2 L298
Pin enB

#define L_S A0 //ir sensor Left
#define R_S A1 //ir sensor Right

#define echo A2       //Echo pin
#define trigger A3 //Trigger pin
```

```
#define servo A5

int Set=15;
int distance_L, distance_F, distance_R;
void setup(){ // put your setup code here, to run once

Serial.begin(9600); // start serial communication at 9600bps

pinMode(R_S, INPUT); // declare if sensor as input pinMode(L_S,
INPUT); // declare ir sensor as input

pinMode(echo, INPUT );// declare ultrasonic sensor Echo pin as input pinMode(trigger,
OUTPUT); // declare ultrasonic sensor Trigger pin as Output

pinMode(enA, OUTPUT); // declare as output for L298 Pin enA
pinMode(in1, OUTPUT); // declare as output for L298 Pin in1
pinMode(in2, OUTPUT); // declare as output for L298 Pin in2
pinMode(in3, OUTPUT); // declare as output for L298 Pin in3
pinMode(in4, OUTPUT); // declare as output for L298 Pin in4
pinMode(enB, OUTPUT); // declare as output for L298 Pin enB

analogWrite(enA, 200); // Write The Duty Cycle 0 to 255 Enable Pin A for Motor1 Speed
analogWrite(enB, 200); // Write The Duty Cycle 0 to 255 Enable Pin B for Motor2 Speed

pinMode(servo, OUTPUT);

 for (int angle = 70; angle <= 140; angle += 5)  { servoPulse(servo, angle);
   }
 for (int angle = 140; angle >= 0; angle -= 5)  {
   servoPulse(servo, angle);  }

 for (int angle = 0; angle <= 70; angle += 5)  {
   servoPulse(servo, angle);  }
```

```
distance_F = Ultrasonic_read();

delay(500);
}

void loop(){
//==============================================
//      Line Follower and Obstacle Avoiding
//==============================================



distance_F = Ultrasonic_read(); Serial.print("D
F=");Serial.println(distance_F);

//if Right Sensor and Left Sensor are at White
color then it will call forword function
 if((digitalRead(R_S) == 0)&&(digitalRead(L_S) == 0)){ if(distance_F >
  Set){forword();}
                 else{Check_side();}
 }

//if Right Sensor is Black and Left Sensor is White then it will call turn Right function
else if((digitalRead(R_S) == 1)&&(digitalRead(L_S) == 0)){turnRight();}

//if Right Sensor is White and Left Sensor is Black then it will call turn Left function
else if((digitalRead(R_S) == 0)&&(digitalRead(L_S) == 1)){turnLeft();}

delay(10);
}

void servoPulse (int pin, int angle){
int pwm = (angle*11) + 500;            // Convert angle to microseconds
 digitalWrite(pin, HIGH);
 delayMicroseconds(pwm); digitalWrite(pin,
```

```
 LOW);
 delay(50); // Refresh cycle of servo
}

//*Ultrasonic_read**************************** long
Ultrasonic_read(){
  digitalWrite(trigger, LOW);
  delayMicroseconds(2); digitalWrite(trigger,
  HIGH); delayMicroseconds(10);
  long time = pulseIn (echo, HIGH); return
  time / 29 / 2;
}

void compareDistance(){
  if(distance_L > distance_R){
  turnLeft();
  delay(500);
  forword();
  delay(600);
  turnRight();
  delay(500);
  forword();
  delay(600);
  turnRight();
  delay(400);
   }
  else{ turnRight();
  delay(500);
  forword();
  delay(600);
  turnLeft();
  delay(500);
  forword();
  delay(600);
```

```
  turnLeft();
  delay(400);
  }
}

void Check_side(){
    Stop(); delay(100);
 for (int angle = 70; angle <= 140; angle += 5)  { servoPulse(servo, angle);
   }
    delay(300);
    distance_R = Ultrasonic_read(); Serial.print("D R=");Serial.println(distance_R);
    delay(100);
  for (int angle = 140; angle >= 0; angle -= 5)  {
   servoPulse(servo, angle);  }
    delay(500);
    distance_L = Ultrasonic_read(); Serial.print("D
    L=");Serial.println(distance_L); delay(100);
 for (int angle = 0; angle <= 70; angle += 5)  {
   servoPulse(servo, angle);  }
    delay(300); compareDistance();
}

void forword(){  //forword
digitalWrite(in1, LOW); //Left Motor backword Pin digitalWrite(in2,
HIGH); //Left Motor forword Pin digitalWrite(in3, HIGH); //Right Motor
forword Pin digitalWrite(in4, LOW); //Right Motor backword Pin
}

void backword(){ //backword

digitalWrite(in1, HIGH); //Left Motor backword Pin digitalWrite(in2, LOW); //Left Motor
forword Pin digitalWrite(in3, LOW); //Right Motor forword Pin digitalWrite(in4, HIGH);
//Right Motor backword Pin
}
```

```
void turnRight(){ //turnRight
digitalWrite(in1, LOW); //Left Motor backword Pin digitalWrite(in2,
HIGH); //Left Motor forword Pin digitalWrite(in3, LOW); //Right Motor
forword Pin digitalWrite(in4, HIGH); //Right Motor backword Pin
}

void turnLeft(){ //turnLeft
digitalWrite(in1, HIGH); //Left Motor backword Pin digitalWrite(in2,
LOW); //Left Motor forword Pin digitalWrite(in3, HIGH); //Right Motor
forword Pin digitalWrite(in4, LOW); //Right Motor backword Pin
}

void Stop(){ //stop
digitalWrite(in1, LOW); //Left Motor backword Pin digitalWrite(in2,
LOW); //Left Motor forword Pin
  digitalWrite(in3, LOW); //Right Motor forword Pin
  digitalWrite(in4, LOW); //Right Motor backword Pin
  }
```

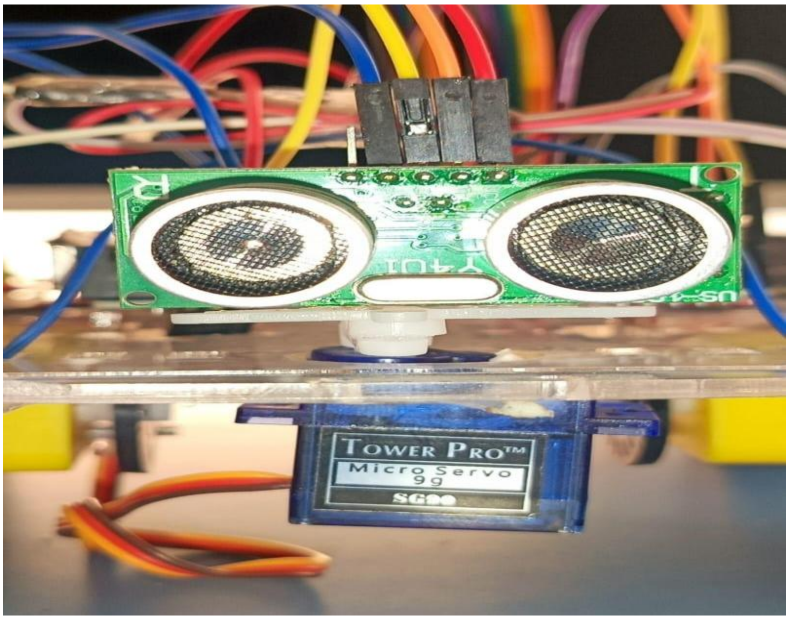


Fig no – 4.2

ULTRASONIC SENSOR

Chapter 5: RESULTS & DISCUSSION

5.1 Testing Scenarios

- **Straight Line Track**: The car followed the line with high accuracy, validating the precise control of its line-following system.

- **Curved Path**: It achieved smooth turning, which proved the system's ability to navigate more complex, non-linear routes.

- **Obstacle Within 20 cm**: The car successfully stopped and avoided a single obstacle, confirming the effectiveness of its ultrasonic sensor and avoidance logic.

- **Multiple Obstacles**: The car was able to avoid multiple obstacles sequentially, demonstrating its reliable performance in a dynamic environment before returning to the line.

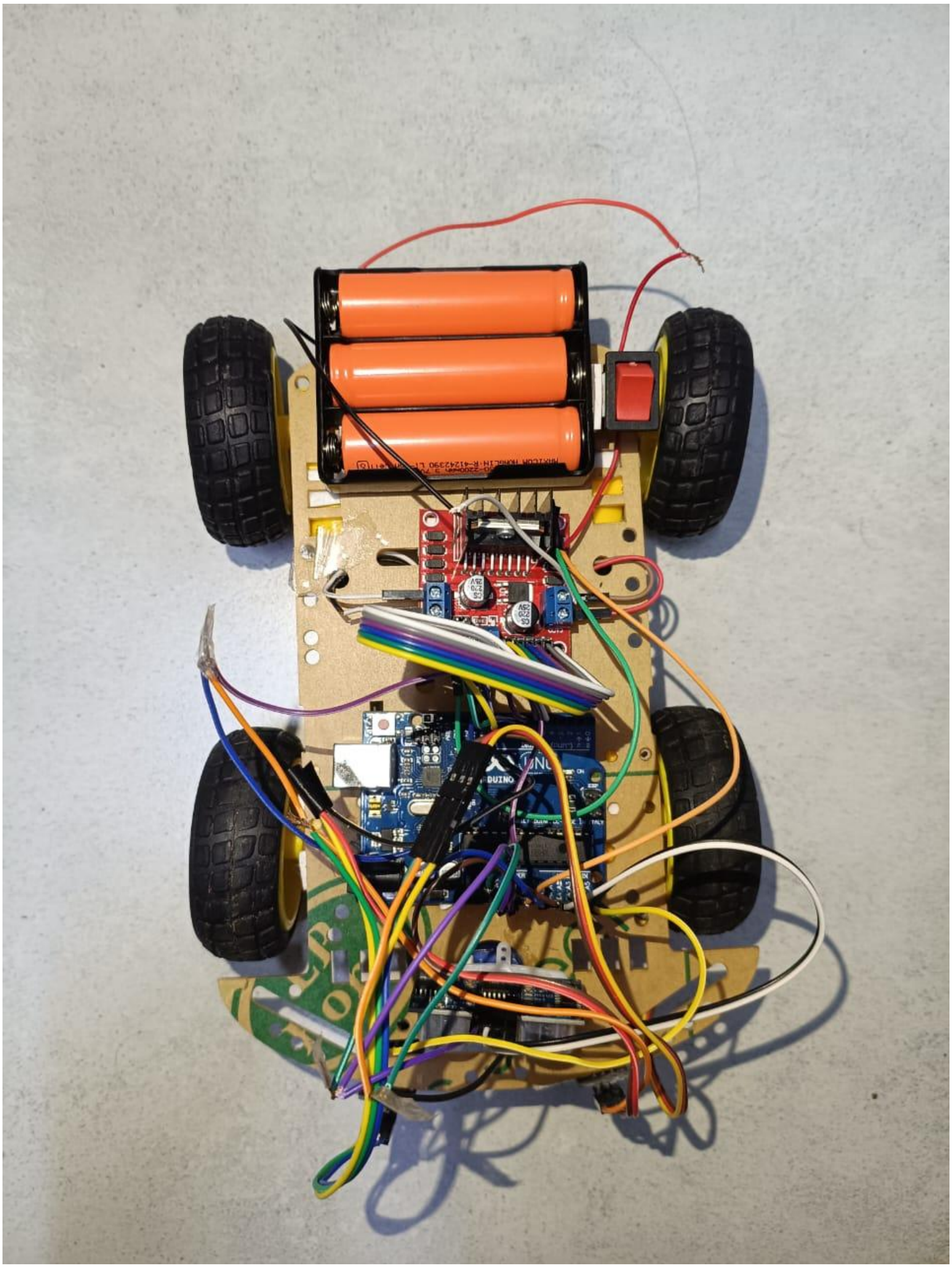


Fig no - 5.1

Testing & Result

5.2 Performance Analysis

- The robot demonstrated a high line-following accuracy of **90–95%** when tested on a white background with black tape. This high percentage indicates that the system's IR sensors and control logic were highly effective at keeping the robot on its intended path with only minimal, momentary deviation.

- The ultrasonic sensor was found to be effective within a range of **2–30 cm**. This is a crucial metric, as it defines the robot's "safe zone." The 2 cm minimum range means the sensor can detect objects very close to the robot, while the 30 cm maximum provides a sufficient warning distance for the robot to initiate an avoidance maneuver..

- The robot operated at a **moderate speed**, which was suitable for a demonstration. This indicates that the primary focus of the design was not on speed but on **stability and reliability**.

5.3 Discussion

The system worked effectively in indoor conditions. However, IR sensors were slightly less accurate under direct sunlight. Ultrasonic sensor performance was good for medium-range obstacles but less effective for very small objects.

Chapter 6: CONCLUSION & FUTURE SCOPE

6.1 Conclusion

- **Successful Dual-Feature Integration**

The project's main achievement was the successful construction of an Arduino-based car that could perform **both line following and obstacle avoidance**. This proved that the limitations of single-function robots can be overcome. By creating a single platform with integrated capabilities, the project delivered a more versatile and practical solution suitable for a wider range of real-world applications, from warehouse logistics to educational demonstrations.

- **Cost-Effective and Accessible Design**

A key objective was to achieve this functionality at a **minimal cost**. The project successfully met this by utilizing affordable, off-the-shelf components like the Arduino Uno and readily available sensors. This not only makes the design accessible for students and hobbyists but also shows that complex robotic behavior can be created without expensive, specialized hardware. The successful **hardware and software integration** further validated the project, as the code efficiently managed the sensors and motors to achieve the desired performance.

- **Confirmed Reliable Performance**

The final testing validated the robot's design and programming. The car demonstrated **reliable performance** on both a predefined track and an obstacle course. This proved that the robot's logic was robust enough to handle the challenges of navigation. The consistent results confirmed that the system could effectively switch between line-following and obstacle-avoiding modes as needed, making it a dependable and intelligent autonomous vehicle

6.2 Future Scope

- **Advanced Sensor Integration**

Upgrading the sensors from basic IR and ultrasonic to a **camera with computer vision** would allow for more advanced tracking. Instead of just detecting a black line, the robot could recognize different line types, colours, and even shapes on the track. This would also improve obstacle detection, allowing the robot to identify and classify objects (e.g., a person vs. a chair) instead of simply detecting their presence.

- **Remote Connectivity (IoT)**

Adding an **IoT (Internet of Things) module** would enable remote control and monitoring. This module would allow the robot to connect to the internet, enabling a user to view its status, track its location, or even control it from a distance. For commercial applications like delivery robots, this feature is essential for dispatch, live tracking, and troubleshooting.

- **Adaptive AI Algorithms**

Integrating **AI algorithms for adaptive path planning** would transform the robot from a simple line-follower into a truly intelligent system. Instead of following a fixed path, the robot could learn from its environment and dynamically choose the most efficient route. For example, if its primary path is blocked, the AI could calculate and navigate a new, alternative route to reach its destination.

- **Outdoor Navigation with GPS**

Implementing a **GPS (Global Positioning System) module** would enable the robot to navigate outdoors. The current system is limited to indoor tracks. With GPS, the robot could follow a coordinate-based route, making it capable of tasks like autonomous delivery in a neighborhood or navigating a large campus.

REFERENCES

1. Mahor, S., Singh, A., & Gupta, R. (2010). *Design and development of obstacle avoiding robot*. International Journal of Engineering Research and Technology (IJERT), 2(3), 45–49.

2. Kumar, A., & Sharma, P. (2015). *Implementation of line follower robot for industrial application*. IEEE International Conference on Advances in Computing, Communications and Informatics, 1058–1063.

3. Sharma, P., & Verma, R. (2018). *Smart robotic car using ultrasonic and infrared sensors*. International Journal of Advanced Research in Computer Science (IJARCS), 9(5), 201– 206.

4. Singh, H., Kaur, M., & Joshi, N. (2020). *Arduino based multi- functional autonomous robot*. International Conference on Computing, Power and Communication Technologies (GUCON), IEEE, 1–5.

5. Arduino. (2023). *Arduino UNO R3 Datasheet*. Retrieved from: https://docs.arduino.cc/hardware/uno-rev3

6. HC-SR04 Ultrasonic Sensor Datasheet. (2022). Retrieved from: https://components101.com/

7. SparkFun Electronics. (2021). *Line Following with IR Sensors*. Retrieved from: https://learn.sparkfun.com/tutorials

8. L293D Motor Driver IC Datasheet. (2022). Texas Instruments. Retrieved from: https://www.ti.com/lit/ds/symlink/l293.pdf